

CS683: Advanced Computer Architecture

Programming Assignment 1: 2D Convolution - The Dhurandhar Way

Dheeraj Kumar Maradana Sabil Ahmad
Roll No: 23B0920 Roll No: 23B1057

Abhineet Majety B Shashank
Roll No: 23B0923 Roll No: 23B1040

September 7, 2026

1 Experimental Methodology & Hardware Environment

All benchmarks were executed on a single hardware core pinned via `taskset -c 0`

- **Processor:** 13th Gen Intel Core i5-13500H (Raptor Lake architecture, Performance Core).
- **Memory Hierarchy:** L1 Data Cache: 48 KB, L2 Cache: 2MB, L3 Cache: 18 MB shared.

2 Task 1A.1: Loop Reordering (conv_reorder)

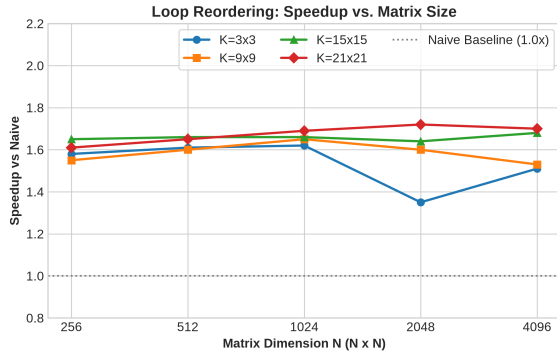
Table 1: Task 1A.1: Performance and Hardware Counters for Loop Reordering vs. Naive

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	256	naive	0.590	1.00×	13.62	5.67	2.40	1.10%	2.06
3	256	reorder	0.373	1.58×	11.78	5.29	2.23	1.07%	2.31
3	512	naive	1.945	1.00×	44.74	13.88	3.22	0.56%	0.98
3	512	reorder	1.205	1.61×	37.44	11.79	3.18	0.57%	1.19
3	1024	naive	7.769	1.00×	169.18	46.11	3.67	0.32%	0.55
3	1024	reorder	4.809	1.62×	139.67	40.08	3.48	0.35%	0.72
3	2048	naive	30.982	1.00×	668.77	184.16	3.63	0.25%	0.42
3	2048	reorder	23.023	1.35×	550.33	178.92	3.08	0.27%	0.55
3	4096	naive	120.201	1.00×	2670.53	719.88	3.71	0.27%	0.46
3	4096	reorder	79.811	1.51×	2200.37	618.50	3.56	0.31%	0.64
9	256	naive	4.254	1.00×	45.49	13.76	3.31	0.25%	0.65
9	256	reorder	2.749	1.55×	45.11	10.60	4.26	0.23%	0.62
9	512	naive	16.665	1.00×	172.26	45.48	3.79	0.10%	0.28
9	512	reorder	10.394	1.60×	170.20	33.02	5.15	0.10%	0.27
9	1024	naive	66.297	1.00×	679.20	173.05	3.92	0.06%	0.17
9	1024	reorder	40.199	1.65×	669.59	121.19	5.53	0.08%	0.21
9	2048	naive	256.935	1.00×	2708.32	671.89	4.03	0.06%	0.15
9	2048	reorder	160.613	1.60×	2668.54	485.17	5.50	0.08%	0.22

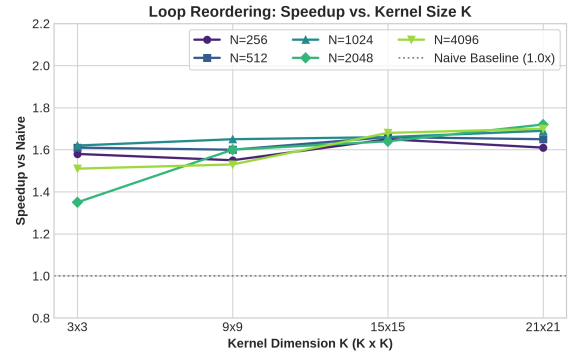
Continued on next page. . .

Table 1 Continued from previous page

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
9	4096	naive	988.595	1.00 $\times$	10833.21	2577.66	4.20	0.06%	0.15
9	4096	reorder	644.248	1.53$\times$	10668.09	1925.36	5.54	0.07%	0.19
15	256	naive	11.903	1.00 $\times$	105.79	30.88	3.43	0.11%	0.32
15	256	reorder	7.200	1.65$\times$	111.74	21.78	5.13	0.10%	0.27
15	512	naive	46.747	1.00 $\times$	413.28	114.39	3.61	0.05%	0.15
15	512	reorder	28.221	1.66$\times$	435.61	75.17	5.80	0.05%	0.13
15	1024	naive	184.457	1.00 $\times$	1643.05	447.31	3.67	0.03%	0.10
15	1024	reorder	111.179	1.66$\times$	1729.26	285.42	6.06	0.06%	0.16
15	2048	naive	740.031	1.00 $\times$	6564.40	1794.02	3.66	0.03%	0.09
15	2048	reorder	450.438	1.64$\times$	6904.61	1131.85	6.10	0.04%	0.12
15	4096	naive	2978.306	1.00 $\times$	26263.96	7588.42	3.46	0.03%	0.09
15	4096	reorder	1777.786	1.68$\times$	27599.80	4508.96	6.12	0.04%	0.10
21	256	naive	23.266	1.00 $\times$	194.43	56.86	3.42	0.06%	0.18
21	256	reorder	14.451	1.61$\times$	211.65	38.28	5.53	0.06%	0.16
21	512	naive	91.597	1.00 $\times$	767.70	218.47	3.51	0.03%	0.09
21	512	reorder	55.400	1.65$\times$	833.67	137.46	6.06	0.05%	0.13
21	1024	naive	371.380	1.00 $\times$	3060.65	864.84	3.54	0.02%	0.07
21	1024	reorder	220.228	1.69$\times$	3318.47	530.93	6.25	0.04%	0.11
21	2048	naive	1476.858	1.00 $\times$	12238.31	3980.04	3.07	0.02%	0.07
21	2048	reorder	858.982	1.72$\times$	13255.69	2110.55	6.28	0.03%	0.09
21	4096	naive	5942.470	1.00 $\times$	48935.37	13855.24	3.53	0.02%	0.06
21	4096	reorder	3503.199	1.70$\times$	52997.99	8458.66	6.27	0.03%	0.08



(a) Reorder Speedup vs. Matrix Size
($K \in \{3, 9, 15, 21\}$)



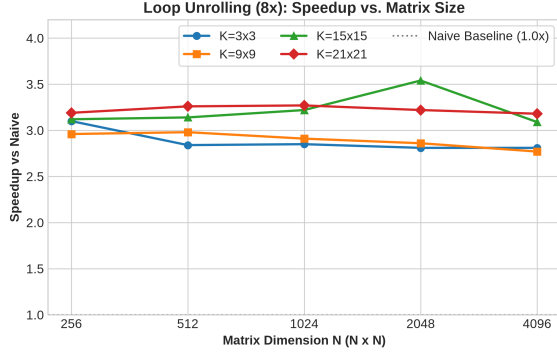
(b) Reorder Speedup vs. Kernel Size
($N \in \{256, 512, 1024, 2048, 4096\}$)

Figure 1: Task 1A.1: Performance Evaluation of Loop Reordering.

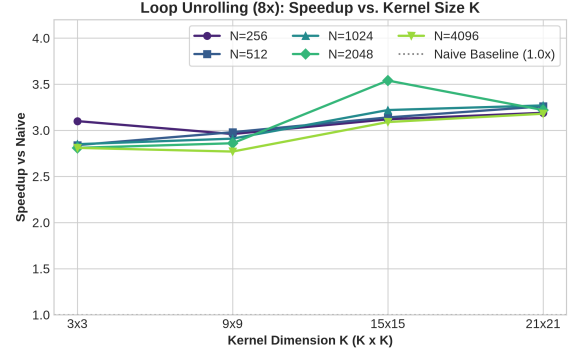
3 Task 1A.2: Loop Unrolling Optimization (conv_unroll)

Table 2: Task 1A.2: Performance and Hardware Counters for Loop Unrolling vs. Naive Baseline

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	256	naive	0.590	1.00 $\times$	13.62	5.67	2.40	1.10%	2.06
3	256	unroll	0.186	3.10$\times$	9.03	4.52	2.00	1.35%	3.05
3	512	naive	1.945	1.00 $\times$	44.74	13.88	3.22	0.56%	0.98
3	512	unroll	0.685	2.84$\times$	26.45	11.15	2.37	0.67%	1.45
3	1024	naive	7.769	1.00 $\times$	169.18	46.11	3.67	0.32%	0.55
3	1024	unroll	2.716	2.85$\times$	96.07	34.71	2.77	0.46%	0.99
3	2048	naive	30.982	1.00 $\times$	668.77	184.16	3.63	0.25%	0.42
3	2048	unroll	11.024	2.81$\times$	376.21	140.08	2.69	0.35%	0.74
3	4096	naive	120.201	1.00 $\times$	2670.53	719.88	3.71	0.27%	0.46
3	4096	unroll	43.049	2.81$\times$	1504.52	534.78	2.81	0.40%	0.85
9	256	naive	4.254	1.00 $\times$	45.49	13.76	3.31	0.25%	0.65
9	256	unroll	1.435	2.96$\times$	17.16	6.99	2.45	0.39%	1.67
9	512	naive	16.665	1.00 $\times$	172.26	45.48	3.79	0.10%	0.28
9	512	unroll	5.683	2.98$\times$	58.97	21.02	2.80	0.18%	0.85
9	1024	naive	66.297	1.00 $\times$	679.20	173.05	3.92	0.06%	0.17
9	1024	unroll	22.768	2.91$\times$	226.09	75.35	3.00	0.13%	0.59
9	2048	naive	256.935	1.00 $\times$	2708.32	671.89	4.03	0.06%	0.15
9	2048	unroll	89.955	2.86$\times$	896.66	304.16	2.95	0.11%	0.51
9	4096	naive	988.595	1.00 $\times$	10833.21	2577.66	4.20	0.06%	0.15
9	4096	unroll	358.717	2.77$\times$	3586.28	1209.45	2.97	0.11%	0.50
15	256	naive	11.903	1.00 $\times$	105.79	30.88	3.43	0.11%	0.32
15	256	unroll	3.805	3.12$\times$	32.97	12.23	2.70	0.16%	0.86
15	512	naive	46.747	1.00 $\times$	413.28	114.39	3.61	0.05%	0.15
15	512	unroll	14.764	3.14$\times$	122.22	42.08	2.90	0.08%	0.45
15	1024	naive	184.457	1.00 $\times$	1643.05	447.31	3.67	0.03%	0.10
15	1024	unroll	59.698	3.22$\times$	478.97	161.03	2.97	0.11%	0.62
15	2048	naive	740.031	1.00 $\times$	6564.40	1794.02	3.66	0.03%	0.09
15	2048	unroll	240.185	3.54$\times$	1909.49	646.54	2.95	0.10%	0.58
15	4096	naive	2978.306	1.00 $\times$	26263.96	7588.42	3.46	0.03%	0.09
15	4096	unroll	952.306	3.09$\times$	7633.13	2597.49	2.94	0.07%	0.40
21	256	naive	23.266	1.00 $\times$	194.43	56.86	3.42	0.06%	0.18
21	256	unroll	7.215	3.19$\times$	56.48	20.07	2.81	0.09%	0.53
21	512	naive	91.597	1.00 $\times$	767.70	218.47	3.51	0.03%	0.09
21	512	unroll	28.433	3.26$\times$	216.18	72.75	2.97	0.06%	0.34
21	1024	naive	371.380	1.00 $\times$	3060.65	864.84	3.54	0.02%	0.07
21	1024	unroll	122.232	3.27$\times$	855.34	331.65	2.58	0.03%	0.20
21	2048	naive	1476.858	1.00 $\times$	12238.31	3980.04	3.07	0.02%	0.07
21	2048	unroll	460.870	3.22$\times$	3412.03	1143.02	2.99	0.02%	0.15
21	4096	naive	5942.470	1.00 $\times$	48935.37	13855.24	3.53	0.02%	0.06
21	4096	unroll	1860.824	3.18$\times$	13644.83	4607.19	2.96	0.03%	0.16

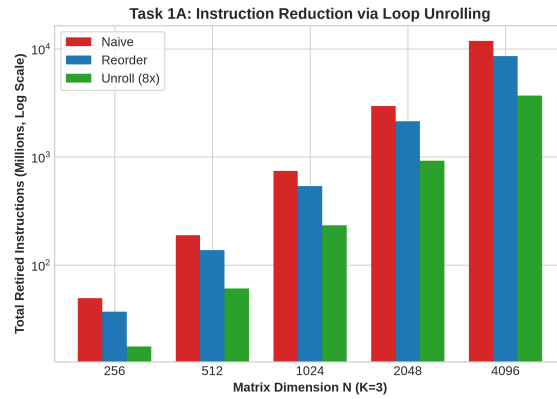


(a) Unroll Speedup vs. Matrix Size
($K \in \{3, 9, 15, 21\}$)



(b) Unroll Speedup vs. Kernel Size
($N \in \{256, 512, 1024, 2048, 4096\}$)

Figure 2: Task 1A.2: Performance Evaluation of Loop Unrolling.



Instruction Count Impact Comparison

Figure 3: Task 1A: Microarchitectural Metric Shifts Under Loop Reordering.

4 Task 1B: Cache Tiling Optimization (conv_tile)

4.1 Deliverables

4.1.1 Profiling baseline 2D convolution code

- L1-D cache : 48 KB
- MPKI : 0.36

4.1.2 Implementing Tiled 2D convolution

All combinations gave poor results, among these 16x256 was better from the data, but since the range of values are too less, it is because of variance, 16x256 is better

Table 3: Task 1B: Performance and Hardware Counters for Cache Tiling Configurations vs. Naive Baseline

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	256	naive	0.289	1.00×	13.53	5.33	2.54	1.03%	1.93
3	256	16 × 16	0.332	0.87×	13.95	5.72	2.44	0.97%	1.87
3	256	16 × 64	0.296	0.98×	13.89	5.53	2.51	1.02%	1.94
3	256	16 × 256	0.283	1.02 ×	13.72	5.44	2.52	1.00%	1.90
3	256	32 × 32	0.304	0.94×	13.77	5.39	2.55	0.99%	1.88
3	256	32 × 64	0.291	1.00 ×	13.74	5.37	2.56	1.01%	1.92
3	256	32 × 128	0.284	1.02 ×	13.72	5.44	2.52	1.04%	1.98
3	256	32 × 256	0.281	1.03 ×	13.72	5.27	2.61	0.98%	1.87
3	256	64 × 64	0.291	0.99×	13.89	5.57	2.49	1.09%	2.08
3	256	64 × 128	0.284	1.01 ×	13.75	5.53	2.48	1.04%	1.98
3	256	64 × 256	0.281	1.02 ×	13.75	5.40	2.54	1.08%	2.06
3	256	128 × 128	0.284	1.01 ×	13.73	5.42	2.53	1.03%	1.95
3	512	naive	1.122	1.00×	44.63	13.54	3.30	0.50%	0.86
3	512	16 × 16	1.332	0.85×	45.88	14.09	3.26	0.52%	0.93
3	512	16 × 64	1.170	0.96×	45.51	13.56	3.36	0.50%	0.88
3	512	16 × 256	1.130	1.00 ×	45.39	13.48	3.37	0.48%	0.86
3	512	32 × 32	1.223	0.92×	45.67	13.87	3.29	0.59%	1.05
3	512	32 × 64	1.156	0.97×	45.49	13.62	3.34	0.57%	1.02
3	512	32 × 128	1.142	0.99×	45.54	13.56	3.36	0.52%	0.93
3	512	32 × 256	1.127	0.99×	45.41	13.44	3.38	0.51%	0.91
3	512	64 × 64	1.170	0.96×	45.56	13.60	3.35	0.55%	0.98
3	512	64 × 128	1.142	0.98×	45.43	13.42	3.39	0.52%	0.93
3	512	64 × 256	1.128	0.99×	45.51	13.60	3.35	0.50%	0.89
3	512	128 × 128	1.141	0.98×	45.43	13.60	3.34	0.53%	0.95
3	1024	naive	4.454	1.00×	169.33	46.71	3.63	0.32%	0.54
3	1024	16 × 16	5.334	0.84×	173.95	48.38	3.60	0.53%	0.94
3	1024	16 × 64	4.658	0.96×	172.50	46.27	3.73	0.35%	0.61
3	1024	16 × 256	4.492	0.99×	172.08	46.04	3.74	0.30%	0.53
3	1024	32 × 32	4.871	0.92×	172.97	47.37	3.65	0.41%	0.72
3	1024	32 × 64	4.718	0.95×	172.47	46.53	3.71	0.34%	0.60
3	1024	32 × 128	4.548	0.98×	172.17	46.22	3.73	0.31%	0.55
3	1024	32 × 256	4.508	0.99×	172.13	46.11	3.73	0.31%	0.54
3	1024	64 × 64	4.666	0.95×	172.49	46.68	3.70	0.35%	0.61
3	1024	64 × 128	4.561	0.98×	172.11	47.46	3.63	0.32%	0.56
3	1024	64 × 256	4.505	0.99×	172.05	46.13	3.73	0.30%	0.53
3	1024	128 × 128	4.546	0.98×	172.19	46.58	3.70	0.31%	0.55
3	2048	naive	17.823	1.00×	669.03	182.72	3.66	0.22%	0.36
3	2048	16 × 16	23.049	0.85×	688.01	204.98	3.36	0.48%	0.84
3	2048	16 × 64	18.821	0.95×	681.87	187.21	3.64	0.28%	0.49

Continued on next page...

Table 3 Continued from previous page

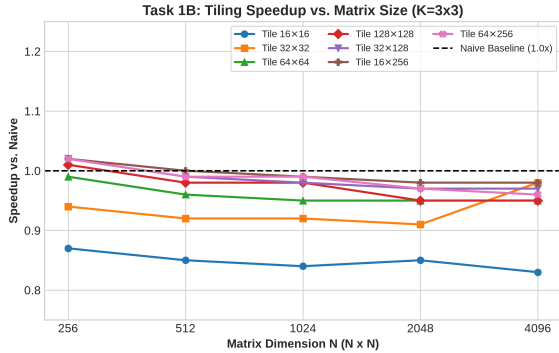
K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	2048	16×256	18.180	$0.98\times$	680.38	184.43	3.69	0.24%	0.41
3	2048	32×32	19.694	$0.91\times$	683.83	188.62	3.63	0.38%	0.65
3	2048	32×64	18.820	$0.95\times$	681.76	186.29	3.66	0.28%	0.48
3	2048	32×128	18.430	$0.97\times$	680.92	186.04	3.66	0.26%	0.44
3	2048	32×256	18.270	$0.98\times$	680.33	187.02	3.64	0.24%	0.41
3	2048	64×64	18.866	$0.95\times$	682.05	188.74	3.61	0.29%	0.50
3	2048	64×128	18.839	$0.95\times$	680.86	187.52	3.63	0.24%	0.42
3	2048	64×256	18.366	$0.97\times$	680.40	186.28	3.65	0.24%	0.41
3	2048	128×128	18.745	$0.95\times$	680.77	189.40	3.59	0.24%	0.42
3	4096	naive	71.501	$1.00\times$	2671.26	710.79	3.76	0.27%	0.45
3	4096	16×16	85.307	$0.83\times$	2753.68	763.86	3.60	0.48%	0.85
3	4096	16×64	74.820	$0.95\times$	2729.09	739.76	3.69	0.28%	0.48
3	4096	16×256	72.550	$0.98\times$	2722.54	750.08	3.63	0.24%	0.41
3	4096	32×32	78.432	$0.98\times$	2736.25	752.14	3.64	0.35%	0.61
3	4096	32×64	75.064	$0.95\times$	2728.40	745.98	3.66	0.27%	0.47
3	4096	32×128	73.495	$0.97\times$	2724.17	740.24	3.68	0.24%	0.42
3	4096	32×256	72.709	$0.98\times$	2723.51	739.89	3.68	0.23%	0.40
3	4096	64×64	75.421	$0.95\times$	2728.96	754.58	3.62	0.27%	0.47
3	4096	64×128	75.929	$0.94\times$	2725.23	751.57	3.63	0.25%	0.42
3	4096	64×256	73.808	$0.96\times$	2722.24	743.39	3.66	0.23%	0.39
3	4096	128×128	75.131	$0.95\times$	2724.18	747.67	3.64	0.24%	0.42
9	256	naive	2.454	$1.00\times$	45.44	14.94	3.04	0.24%	0.62
9	256	16×16	2.421	$1.02\times$	46.09	13.53	3.41	0.23%	0.59
9	256	16×64	2.413	$1.02\times$	46.07	13.69	3.37	0.22%	0.59
9	256	16×256	2.414	$1.02\times$	46.09	13.34	3.45	0.23%	0.60
9	256	32×32	2.423	$1.02\times$	46.07	13.56	3.40	0.23%	0.59
9	256	32×64	2.411	$1.02\times$	46.05	13.67	3.37	0.24%	0.62
9	256	32×128	2.418	$1.02\times$	46.11	13.60	3.39	0.23%	0.60
9	256	32×256	2.414	$1.02\times$	46.02	13.58	3.39	0.23%	0.61
9	256	64×64	2.411	$1.02\times$	46.03	13.62	3.38	0.23%	0.61
9	256	64×128	2.413	$1.02\times$	46.01	13.47	3.42	0.24%	0.62
9	256	64×256	2.409	$1.02\times$	46.11	13.58	3.39	0.24%	0.62
9	256	128×128	2.412	$1.02\times$	46.01	13.44	3.42	0.24%	0.63
9	512	naive	9.792	$1.00\times$	172.09	45.92	3.75	0.10%	0.25
9	512	16×16	9.674	$1.01\times$	175.00	46.53	3.76	0.09%	0.24
9	512	16×64	9.649	$1.01\times$	174.64	46.20	3.78	0.10%	0.27
9	512	16×256	9.611	$1.02\times$	174.55	46.20	3.78	0.10%	0.27
9	512	32×32	9.674	$1.01\times$	174.73	46.18	3.78	0.09%	0.25
9	512	32×64	9.650	$1.01\times$	174.63	46.24	3.78	0.10%	0.27
9	512	32×128	9.645	$1.01\times$	174.54	46.04	3.79	0.10%	0.27
9	512	32×256	9.613	$1.02\times$	174.53	46.03	3.79	0.10%	0.27
9	512	64×64	9.639	$1.02\times$	174.59	46.46	3.76	0.10%	0.27
9	512	64×128	9.628	$1.02\times$	174.55	46.11	3.79	0.10%	0.28
9	512	64×256	9.640	$1.02\times$	174.53	46.00	3.79	0.10%	0.27
9	512	128×128	9.639	$1.01\times$	174.54	46.22	3.78	0.10%	0.27
9	1024	naive	38.932	$1.00\times$	678.78	173.65	3.91	0.05%	0.14
9	1024	16×16	38.646	$1.01\times$	690.24	178.12	3.88	0.05%	0.14
9	1024	16×64	39.887	$0.98\times$	688.70	193.60	3.56	0.08%	0.20
9	1024	16×256	41.166	$1.03\times$	688.31	189.39	3.63	0.07%	0.18
9	1024	32×32	39.035	$1.08\times$	689.28	177.65	3.88	0.07%	0.18
9	1024	32×64	38.419	$1.01\times$	688.61	174.52	3.95	0.07%	0.20
9	1024	32×128	38.110	$1.02\times$	688.45	175.14	3.93	0.07%	0.18
9	1024	32×256	37.876	$1.03\times$	688.31	175.15	3.93	0.06%	0.17
9	1024	64×64	38.400	$1.02\times$	688.73	176.35	3.91	0.07%	0.19
9	1024	64×128	38.194	$1.02\times$	688.51	174.79	3.94	0.07%	0.17
9	1024	64×256	38.020	$1.03\times$	688.48	174.98	3.93	0.06%	0.17

Continued on next page...

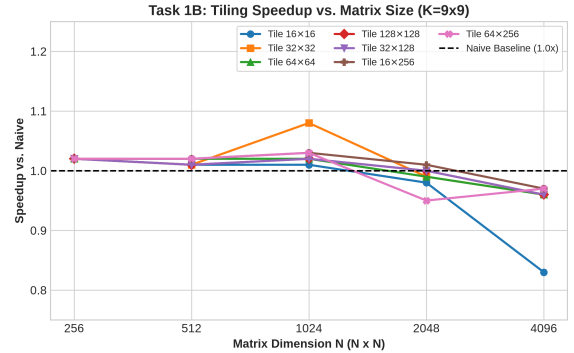
Table 3 Continued from previous page

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
9	1024	128×128	38.291	1.02 $\times$	688.49	176.05	3.91	0.07%	0.17
9	2048	naive	151.983	1.00 $\times$	2708.33	684.28	3.96	0.05%	0.13
9	2048	16×16	154.881	0.98 $\times$	2753.13	717.04	3.84	0.04%	0.12
9	2048	16×64	153.359	0.99 $\times$	2747.86	711.41	3.86	0.07%	0.18
9	2048	16×256	150.134	1.01 $\times$	2745.73	695.42	3.95	0.06%	0.15
9	2048	32×32	153.955	0.99 $\times$	2749.19	713.82	3.85	0.06%	0.15
9	2048	32×64	153.410	0.99 $\times$	2747.47	708.26	3.88	0.06%	0.17
9	2048	32×128	152.134	1.00 $\times$	2745.85	705.59	3.89	0.06%	0.15
9	2048	32×256	150.712	1.00 $\times$	2746.07	698.10	3.93	0.05%	0.14
9	2048	64×64	153.501	0.99 $\times$	2747.53	760.39	3.61	0.06%	0.16
9	2048	64×128	162.497	1.02 $\times$	2746.67	763.71	3.60	0.06%	0.15
9	2048	64×256	173.895	0.95 $\times$	2747.30	883.21	3.11	0.06%	0.15
9	2048	128×128	152.422	1.00 $\times$	2747.09	705.22	3.90	0.06%	0.15
9	4096	naive	592.435	1.00 $\times$	10833.28	2749.11	3.94	0.05%	0.13
9	4096	16×16	714.681	0.83 $\times$	11017.71	3157.55	3.49	0.05%	0.12
9	4096	16×64	652.294	1.05 $\times$	10992.00	3147.30	3.49	0.07%	0.17
9	4096	16×256	605.487	0.97 $\times$	10984.61	2796.81	3.93	0.06%	0.15
9	4096	32×32	615.722	0.96 $\times$	10996.53	2845.98	3.86	0.06%	0.16
9	4096	32×64	614.030	0.96 $\times$	10989.69	2829.98	3.88	0.06%	0.17
9	4096	32×128	611.434	0.96 $\times$	10986.10	2822.86	3.89	0.06%	0.15
9	4096	32×256	607.456	0.97 $\times$	10982.58	2799.84	3.92	0.05%	0.14
9	4096	64×64	614.358	0.96 $\times$	10989.71	2837.98	3.87	0.06%	0.16
9	4096	64×128	611.966	0.97 $\times$	10985.21	2825.71	3.89	0.05%	0.14
9	4096	64×256	607.961	0.97 $\times$	10983.68	2809.89	3.91	0.05%	0.14
9	4096	128×128	613.306	0.96 $\times$	10985.94	2894.80	3.80	0.06%	0.15

4.1.3 Collecting and analyzing different metrics of interest



(a) Tiling Speedup vs. Matrix Size ($K = 3$)

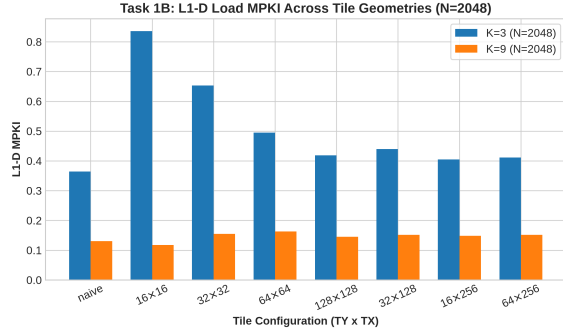


(b) Tiling Speedup vs. Matrix Size ($K = 9$)

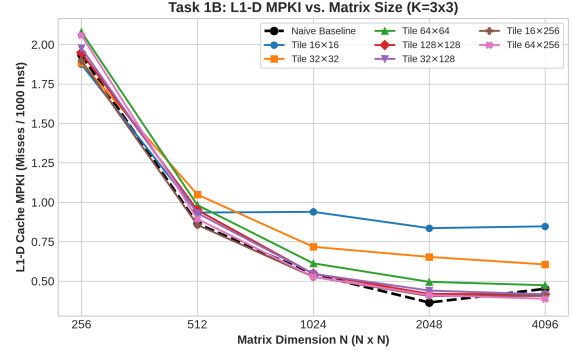
Figure 4: Task 1B: Performance Evaluation Across Cache Tiling Configurations.

4.1.4 Question 1: L1D-MPKI Tiling vs Naive

- There is no clear pattern, in some configurations MPKI decreased but in most cases it increases leading to negative speedup
- The L1D miss rate is $< 0.5\%$ so tiling isn't really helping, worsen performance due to constant factors



(a) L1-D Miss Rate Comparison



(b) L1-D MPKI Comparison ($K = 3$)

Figure 5: Task 1B: Cache Memory Hierarchy Behavior Under Tiling.

4.1.5 Question 2: L1-D MPKI Variation Across Matrix Sizes and Tile Sizes

- **Variation Across Matrix Sizes (N):**

- L1D-MPKI steadily decreased with Matrix Size
- At small N , cold compulsory misses dominate per instruction. At large N , the working set fits in cache keeping the MPKI small

- **Variation Across Tile Sizes:**

- For $K=3$, there is a decreasing pattern of L1-D MPKI but there is no clear pattern for $K=9$
- The general decreasing pattern is due to better utilization of L1D cache

4.1.6 Question 3: Speedup Evaluation, Limiting Factors, and Proposed Solutions

- **Speedup Quantification:**

- Tiling alone yields **no significant speedup**, fluctuating tightly between $0.83\times$ and $1.02\times$ across all workloads.

- **Limiting Factors:**

- Unlike matrix multiplication where reuse scales as $O(N)$, 2D convolution data reuse is capped at K^2 . The active rows already fit in the cache in naive code, hence tiling isn't improving

- **Proposed Solution:**

- Combine cache tiling with **AVX2 SIMD** (to execute 8 FMAs per cycle), so cache usage becomes more, as implemented in Task 1D.

5 Task 1C: SIMD Vectorization Optimization (conv_simd)

5.1 Deliverables

1. Baseline Profiling

- Instructions : 667.59M

3. Instruction Count and Performance Analysis SIMD Implementations:

- **128-bit SIMD (conv_simd128)**: Utilizes Intel SSE/AVX 128-bit vector registers (`_mm128`) to process 4 single-precision floating-point numbers concurrently per vector operation.
 - Instructions: 371.71M
 - Speedup: 4.06x
- **256-bit SIMD (conv_simd256)**: Utilizes Intel AVX2 256-bit vector registers (`_mm256`) to process 8 single-precision floating-point numbers concurrently per vector operation.
 - Instructions: 328.82M
 - Speedup: 7.81x
- **512-bit SIMD** is not supported on my Laptop

4. Multi-Size Evaluation

Table 4 summarizes the execution time, speedup, instruction counts, cycles, IPC, L1-D miss rates, and L1-D MPKI across all evaluated matrix sizes ($N \in \{256, 512, 1024, 2048, 4096\}$) and kernel sizes ($K \in \{3, 9\}$).

Table 4: Task 1C: Performance and Hardware Counters for SIMD Vectorization (128-bit & 256-bit) vs. Naive Baseline

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	256	naive	0.311	1.00×	13.56	5.15	2.63	1.42%	2.67
3	256	simd128 (SSE)	0.069	4.51 ×	8.91	4.61	1.93	1.56%	2.90
3	256	simd256 (AVX2)	0.036	8.64 ×	8.24	4.48	1.84	1.71%	3.13
3	512	naive	1.122	1.00×	44.73	13.29	3.37	0.54%	0.94
3	512	simd128 (SSE)	0.268	4.19 ×	26.20	10.21	2.57	0.93%	1.51
3	512	simd256 (AVX2)	0.129	8.70 ×	23.41	9.70	2.41	1.08%	1.68
3	1024	naive	4.461	1.00×	168.85	44.98	3.75	0.29%	0.49
3	1024	simd128 (SSE)	1.055	4.23 ×	94.92	32.97	2.88	0.59%	0.92
3	1024	simd256 (AVX2)	0.503	8.87 ×	84.17	31.29	2.69	0.74%	1.07
3	2048	naive	17.797	1.00×	667.59	180.68	3.69	0.21%	0.35
3	2048	simd128 (SSE)	4.387	4.06 ×	371.71	133.92	2.78	0.46%	0.69
3	2048	simd256 (AVX2)	2.278	7.81 ×	328.82	125.37	2.62	0.57%	0.81
3	4096	naive	71.334	1.00×	2670.81	703.13	3.80	0.23%	0.39
3	4096	simd128 (SSE)	17.889	3.99 ×	1488.09	508.25	2.93	0.50%	0.75
3	4096	simd256 (AVX2)	9.806	7.27 ×	1315.27	484.29	2.72	0.65%	0.92
9	256	naive	2.459	1.00×	45.52	13.35	3.41	0.23%	0.62
9	256	simd128 (SSE)	0.751	3.27 ×	15.78	7.11	2.22	0.70%	1.79
9	256	simd256 (AVX2)	0.376	6.54 ×	11.69	5.60	2.09	0.95%	2.19
9	512	naive	9.785	1.00×	172.23	45.60	3.78	0.10%	0.25
9	512	simd128 (SSE)	2.985	3.28 ×	53.38	20.24	2.64	0.34%	0.87
9	512	simd256 (AVX2)	1.504	6.51 ×	37.02	14.54	2.55	0.45%	1.02
9	1024	naive	39.016	1.00×	678.88	171.88	3.95	0.05%	0.14
9	1024	simd128 (SSE)	11.934	3.27 ×	203.66	73.50	2.77	0.20%	0.51
9	1024	simd256 (AVX2)	6.015	6.49 ×	138.49	50.83	2.72	0.28%	0.63
9	2048	naive	151.971	1.00×	2707.34	673.27	4.02	0.04%	0.12
9	2048	simd128 (SSE)	47.779	3.18 ×	808.32	299.79	2.70	0.17%	0.43
9	2048	simd256 (AVX2)	24.098	6.31 ×	546.29	207.94	2.63	0.27%	0.61
9	4096	naive	592.072	1.00×	10832.44	2627.20	4.12	0.05%	0.12
9	4096	simd128 (SSE)	191.157	3.10 ×	3228.33	1194.30	2.70	0.16%	0.42

Continued on next page...

Table 4 Continued from previous page

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
9	4096	simd256 (AVX2)	135.740	4.36×	2186.83	835.33	2.62	0.28%	0.63

Task 1C: SIMD Speedup vs. Matrix Size Across Register Widths

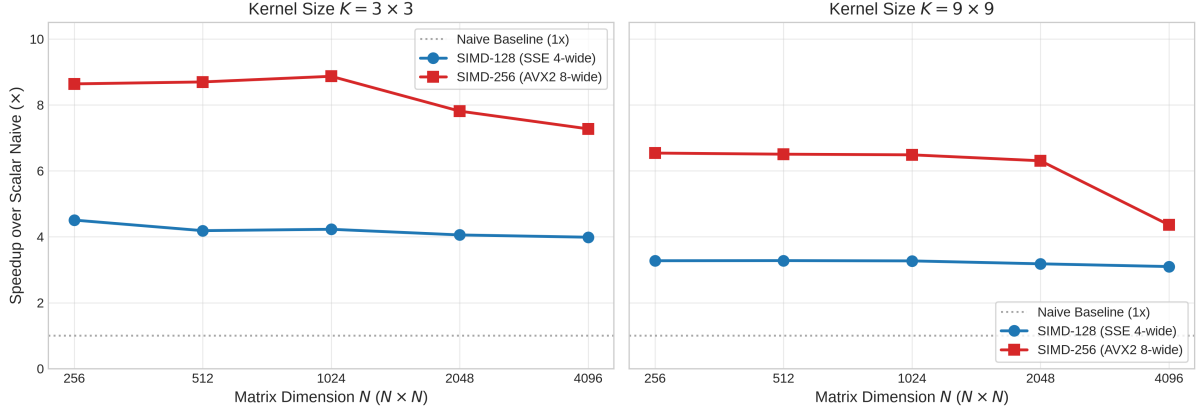
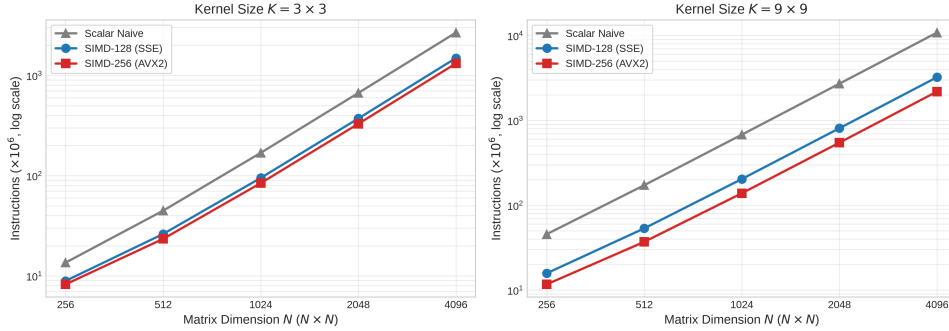


Figure 6: Task 1C: Speedup over Scalar Naive Baseline across Matrix Sizes for SIMD-128 (SSE) and SIMD-256 (AVX2).

Task 1C: Instruction Count Reduction Across SIMD Register Widths



(a) Instructions Reduction Across Register Widths

Figure 7: Task 1C: Microarchitectural Impact of SIMD Vectorization.

5.1.1 Question 1: Change in Number of Instructions from Naive to SIMD Implementation

- **Observed Changes:**

- **SIMD-128 (SSE):** Reduces instructions by **44.3% to 70.1%** compared to naive baseline (e.g., at $N = 2048, K = 3$: $667.59 \times 10^6 \rightarrow 371.71 \times 10^6$ [−44.3%]; at $K = 9$: $2707.34 \times 10^6 \rightarrow 808.32 \times 10^6$ [−70.1%]).
- **SIMD-256 (AVX2):** Reduces instructions by **50.8% to 79.8%** compared to naive baseline (e.g., at $N = 2048, K = 3$: $667.59 \times 10^6 \rightarrow 328.82 \times 10^6$ [−50.8%]; at $K = 9$: $2707.34 \times 10^6 \rightarrow 546.29 \times 10^6$ [−79.8%]).

- **Justifications:**

- Since SIMD is packing some instructions into packs of 4 or 8, the total instructions reduces
- Number of iterations of innermost loop reduces hence number of branch conditional instructions reduces

5.1.2 Question 2: Speedup Evaluation, Contributing Factors

- **Speedup Quantification:**

- **SIMD-128 (SSE):** Got $3.10\times\text{--}4.51\times$ **speedup** over naive across all matrix and kernel dimensions
- **SIMD-256 (AVX2):** Got $4.36\times\text{--}8.87\times$ **speedup** over naive across all matrix and kernel dimensions

- **Contributing Factors:**

- 128-bit registers process 4 single-precision floats simultaneously; 256-bit registers process 8 single-precision floats simultaneously per instruction hence reducing total cycles

5.1.3 Question 3: SIMD Intrinsics Used

1. `_mm_setzero_ps()` / `_mm256_setzero_ps()`:

- Initializes vector accumulator register (`acc`) to zero

2. `_mm_set1_ps(ker[...])` / `_mm256_set1_ps(ker[...])`:

- Broadcasts the kernel coefficient `ker[ky · K + kx]` across all 4 (SSE) or 8 (AVX2) vector lanes (`[w, w, w, w, ...]`).

3. `_mm_loadu_ps(...)` / `_mm256_loadu_ps(...)`:

- Fetches 4 / 8 contiguous input floats from memory starting at address `&in[(oy + ky) · stride + (ox + kx)]`.

4. `_mm_fmadd_ps()` / `_mm256_fmadd_ps(...)`:

- Performs fused multiply-accumulate $acc \leftarrow (in_vec \times k_vec) + acc$ in a single instruction.

5. `_mm_storeu_ps(...)` / `_mm256_storeu_ps(...)`:

- Writes accumulated vector floats back to the output row at `&out[oy · W + ox]`.

6 Task 1D: Sabka saath sabka vikaas (conv_optimized)

- **Limits of Isolated Optimizations:**

- **Loop Reordering (conv_reorder):** Establishes unit-stride memory access and prefetcher engagement, but remains strictly bound by the scalar 4-cycle FMA latency chain ($1.35\times\text{--}1.72\times$).
- **Loop Unrolling (conv_unroll):** Eliminates the 4-cycle FMA latency stall across 8 independent accumulators, but executes scalar instructions (1 float/op), leaving the CPU's 256-bit vector execution pipelines idle ($2.8\times\text{--}3.59\times$).

- **Cache Tiling** (`conv_tile`): Constrains working sets, but cannot eliminate arithmetic dependency latency or scalar execution limits ($0.83\times$ – $1.02\times$).
- **SIMD Vectorization** (`conv_simd`): Processes 8 floats per instruction, but relies on a single vector accumulator (`acc`), causing the processor to stall for 4 cycles between consecutive `_mm256_fmadd_ps` instructions ($4.2\times$ – $8.8\times$).

- **Synergy in `conv_optimized`:**

1. **SIMD + 8-Way Multi-Accumulator Unrolling:** Combines AVX2 256-bit SIMD with **8 independent vector accumulators** (`acc0` through `acc7`, processing 64 floats simultaneously)
2. **Loop-Invariant Pointer Hoisting:** Invariant kernel weight broadcasts (`_mm256_set1_ps`) and row base addresses are hoisted outside the inner accumulator loop.
3. **Cache-Aware Y-Tiling:** For large dimensions ($N \geq 2048$), vertical tiling ($TY = 64$) keeps the active row working set confined within the 2 MB L2 cache, eliminating cache evictions

6.1 Comprehensive Multi-Stage Performance Comparison

Table 5 details the execution time, speedup, instruction counts, cycles, IPC, L1-D miss rates, and L1-D MPKI across all six evaluated implementations for $K \in \{3, 9, 15, 21\}$ and $N \in \{256, 512, 1024, 2048, 4096\}$.

Table 5: Task 1D: Comprehensive Performance Comparison Across All Six Optimization Stages

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	256	naive	0.625	$1.00\times$	13.56	5.17	2.62	1.04%	1.95
3	256	reorder	0.409	$1.53\times$	11.76	4.98	2.36	1.09%	2.35
3	256	unroll	0.174	$3.59\times$	9.06	4.78	1.89	1.32%	2.98
3	256	tile	0.495	$1.26\times$	13.73	5.26	2.61	1.01%	1.92
3	256	simd	0.082	$7.62\times$	8.24	4.21	1.96	1.71%	3.14
3	256	optimized	0.041	$15.24\times$	7.80	4.31	1.81	2.00%	3.74
3	512	naive	1.947	$1.00\times$	44.64	13.02	3.43	0.53%	0.92
3	512	reorder	1.210	$1.61\times$	37.41	11.65	3.21	0.56%	1.17
3	512	unroll	0.686	$2.84\times$	26.41	10.60	2.49	0.72%	1.56
3	512	tile	1.952	$1.00\times$	45.41	13.14	3.46	0.51%	0.91
3	512	simd	0.237	$8.22\times$	23.44	9.46	2.48	1.13%	1.76
3	512	optimized	0.144	$13.52\times$	21.46	9.28	2.31	1.45%	2.31
3	1024	naive	7.719	$1.00\times$	169.37	45.11	3.75	0.32%	0.55
3	1024	reorder	4.906	$1.57\times$	139.64	39.42	3.54	0.34%	0.71
3	1024	unroll	2.752	$2.80\times$	96.00	34.15	2.81	0.43%	0.93
3	1024	tile	7.798	$0.99\times$	172.09	45.12	3.81	0.31%	0.55
3	1024	simd	1.004	$7.69\times$	84.12	34.72	2.42	0.77%	1.12
3	1024	optimized	0.665	$11.61\times$	76.25	32.74	2.33	1.12%	1.65
3	2048	naive	31.197	$1.00\times$	668.76	182.77	3.66	0.24%	0.41
3	2048	reorder	20.487	$1.52\times$	550.38	166.51	3.31	0.27%	0.56
3	2048	unroll	10.972	$2.84\times$	376.14	133.47	2.82	0.37%	0.78
3	2048	tile	31.223	$1.00\times$	680.35	178.19	3.82	0.26%	0.44
3	2048	simd	4.080	$7.65\times$	328.49	118.98	2.76	0.68%	0.97
3	2048	optimized	2.607	$11.97\times$	296.78	115.90	2.56	0.97%	1.39
3	4096	naive	122.314	$1.00\times$	2670.94	696.23	3.84	0.31%	0.53
3	4096	reorder	81.133	$1.51\times$	2200.76	620.36	3.55	0.31%	0.64

Continued on next page...

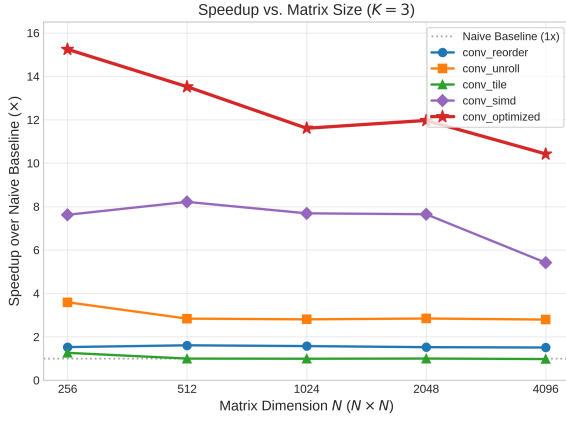
Table 5 Continued from previous page

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
3	4096	unroll	43.769	$2.79\times$	1505.04	536.37	2.81	0.42%	0.89
3	4096	tile	125.451	$0.97\times$	2722.11	715.47	3.80	0.26%	0.45
3	4096	simd	22.582	$5.42\times$	1314.34	472.15	2.78	0.80%	1.13
3	4096	optimized	11.732	$10.43\times$	1188.43	465.41	2.55	1.57%	2.24
9	256	naive	4.246	$1.00\times$	45.45	13.28	3.42	0.24%	0.63
9	256	reorder	2.741	$1.55\times$	45.08	10.20	4.42	0.23%	0.62
9	256	unroll	1.442	$2.94\times$	17.15	6.99	2.45	0.39%	1.68
9	256	tile	4.172	$1.02\times$	46.04	13.31	3.46	0.23%	0.60
9	256	simd	0.651	$6.52\times$	11.67	5.43	2.15	0.99%	2.28
9	256	optimized	0.284	$14.95\times$	8.78	4.53	1.94	1.35%	3.25
9	512	naive	16.939	$1.00\times$	172.24	44.54	3.87	0.10%	0.28
9	512	reorder	10.470	$1.62\times$	170.19	32.27	5.27	0.10%	0.28
9	512	unroll	5.736	$2.95\times$	58.93	20.88	2.82	0.18%	0.82
9	512	tile	16.613	$1.02\times$	174.65	45.53	3.84	0.11%	0.29
9	512	simd	2.605	$6.50\times$	37.04	14.18	2.61	0.50%	1.12
9	512	optimized	1.136	$14.91\times$	25.52	11.04	2.31	0.80%	1.90
9	1024	naive	66.954	$1.00\times$	679.12	171.33	3.96	0.06%	0.16
9	1024	reorder	41.271	$1.62\times$	669.53	120.00	5.58	0.08%	0.21
9	1024	unroll	22.960	$2.92\times$	226.04	75.66	2.99	0.12%	0.57
9	1024	tile	64.813	$1.03\times$	688.61	173.29	3.97	0.07%	0.20
9	1024	simd	10.421	$6.42\times$	138.55	50.08	2.77	0.30%	0.68
9	1024	optimized	5.129	$13.05\times$	92.56	40.04	2.31	0.80%	1.90
9	2048	naive	270.169	$1.00\times$	2709.06	692.15	3.91	0.06%	0.15
9	2048	reorder	164.132	$1.65\times$	2668.99	481.36	5.54	0.08%	0.22
9	2048	unroll	90.561	$2.98\times$	896.86	306.27	2.93	0.11%	0.50
9	2048	tile	254.496	$1.06\times$	2747.39	689.87	3.98	0.07%	0.17
9	2048	simd	59.464	$4.54\times$	546.47	203.12	2.69	0.30%	0.66
9	2048	optimized	26.367	$10.25\times$	362.47	151.58	2.39	0.38%	0.89
9	4096	naive	996.348	$1.00\times$	10834.82	2960.31	3.66	0.06%	0.15
9	4096	reorder	726.037	$1.37\times$	10670.81	2126.09	5.02	0.08%	0.22
9	4096	unroll	400.766	$2.49\times$	3587.30	1316.09	2.73	0.10%	0.49
9	4096	tile	1105.586	$0.90\times$	10989.75	2770.69	3.97	0.06%	0.17
9	4096	simd	244.214	$4.08\times$	2186.24	810.19	2.70	0.29%	0.66
9	4096	optimized	111.227	$8.96\times$	1449.85	608.80	2.38	1.21%	2.84
15	256	naive	11.896	$1.00\times$	105.74	30.66	3.45	0.10%	0.30
15	256	reorder	7.527	$1.58\times$	111.79	21.38	5.23	0.09%	0.26
15	256	unroll	3.812	$3.12\times$	32.95	12.37	2.66	0.16%	0.85
15	256	tile	11.934	$1.00\times$	106.72	30.22	3.53	0.11%	0.31
15	256	simd	2.230	$5.33\times$	18.02	8.81	2.05	0.59%	1.67
15	256	optimized	0.802	$14.83\times$	10.78	5.80	1.86	0.79%	2.53
15	512	naive	47.535	$1.00\times$	413.27	113.79	3.63	0.05%	0.16
15	512	reorder	29.093	$1.63\times$	435.60	74.08	5.88	0.05%	0.14
15	512	unroll	15.177	$3.13\times$	122.22	41.91	2.92	0.08%	0.46
15	512	tile	47.655	$1.00\times$	417.18	113.67	3.67	0.05%	0.16
15	512	simd	8.963	$5.30\times$	62.56	27.88	2.24	0.32%	0.92
15	512	optimized	3.218	$14.77\times$	33.48	15.70	2.13	0.43%	1.47
15	1024	naive	189.862	$1.00\times$	1643.32	447.13	3.68	0.03%	0.10
15	1024	reorder	114.291	$1.66\times$	1729.07	283.97	6.09	0.06%	0.16
15	1024	unroll	61.698	$3.08\times$	479.22	160.97	2.98	0.11%	0.65
15	1024	tile	190.539	$1.00\times$	1659.13	447.58	3.71	0.04%	0.13
15	1024	simd	52.467	$3.62\times$	240.52	104.65	2.30	0.16%	0.45
15	1024	optimized	19.988	$9.50\times$	124.33	57.27	2.17	1.61%	5.58
15	2048	naive	759.045	$1.00\times$	6565.58	1790.71	3.67	0.03%	0.09
15	2048	reorder	460.110	$1.65\times$	6903.90	1128.88	6.12	0.04%	0.12

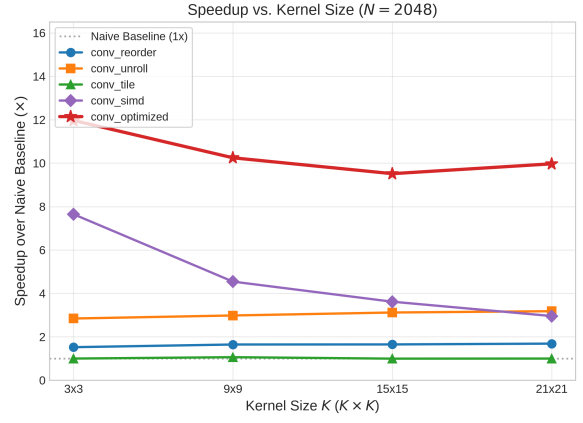
Continued on next page...

Table 5 Continued from previous page

K	N	Configuration	Time (ms)	Speedup	Instructions ($\times 10^6$)	Cycles ($\times 10^6$)	IPC	L1-D Miss Rate	L1-D MPKI
15	2048	unroll	243.285	$3.12\times$	1909.46	646.91	2.95	0.10%	0.55
15	2048	tile	762.873	$0.99\times$	6628.63	1790.77	3.70	0.04%	0.12
15	2048	simd	209.817	$3.62\times$	954.15	418.67	2.28	0.13%	0.36
15	2048	optimized	79.777	$9.51\times$	490.36	231.73	2.12	1.61%	5.60
15	4096	naive	3028.238	$1.00\times$	26257.51	7254.62	3.62	0.03%	0.08
15	4096	reorder	1818.838	$1.66\times$	27600.46	4512.02	6.12	0.04%	0.10
15	4096	unroll	983.226	$3.08\times$	7632.57	2594.48	2.94	0.06%	0.37
15	4096	tile	3041.492	$1.00\times$	26511.90	7171.84	3.70	0.04%	0.11
15	4096	simd	830.062	$3.65\times$	3815.11	1675.68	2.28	0.12%	0.36
15	4096	optimized	318.977	$9.49\times$	1956.84	925.63	2.11	1.62%	5.64
21	256	naive	23.582	$1.00\times$	194.36	57.14	3.40	0.06%	0.18
21	256	reorder	14.665	$1.61\times$	211.58	38.07	5.56	0.06%	0.17
21	256	unroll	7.365	$3.20\times$	56.50	20.21	2.80	0.09%	0.52
21	256	tile	23.624	$1.00\times$	195.78	56.58	3.46	0.06%	0.17
21	256	simd	5.509	$4.28\times$	27.41	16.23	1.69	0.42%	1.31
21	256	optimized	1.565	$15.07\times$	13.73	7.45	1.84	0.51%	2.02
21	512	naive	94.938	$1.00\times$	767.64	218.69	3.51	0.03%	0.09
21	512	reorder	56.447	$1.68\times$	833.61	137.00	6.08	0.05%	0.13
21	512	unroll	29.149	$3.26\times$	216.17	73.12	2.96	0.06%	0.35
21	512	tile	94.714	$1.00\times$	773.09	218.23	3.54	0.04%	0.11
21	512	simd	21.933	$4.33\times$	99.90	57.00	1.75	0.17%	0.55
21	512	optimized	6.282	$15.11\times$	45.32	22.67	2.00	0.34%	1.45
21	1024	naive	379.080	$1.00\times$	3060.54	866.28	3.53	0.02%	0.07
21	1024	reorder	226.277	$1.68\times$	3318.96	529.24	6.27	0.04%	0.12
21	1024	unroll	117.969	$3.21\times$	855.17	285.61	2.99	0.03%	0.18
21	1024	tile	379.335	$1.00\times$	3082.96	865.50	3.56	0.03%	0.09
21	1024	simd	128.080	$2.96\times$	389.94	220.45	1.77	0.10%	0.33
21	1024	optimized	38.184	$9.93\times$	171.59	85.11	2.02	1.10%	4.83
21	2048	naive	1514.396	$1.00\times$	12235.16	3460.83	3.54	0.02%	0.06
21	2048	reorder	897.813	$1.69\times$	13255.97	2099.29	6.31	0.03%	0.08
21	2048	unroll	476.201	$3.18\times$	3412.35	1149.89	2.97	0.03%	0.17
21	2048	tile	1517.017	$1.00\times$	12324.16	3466.85	3.55	0.03%	0.08
21	2048	simd	512.044	$2.96\times$	1551.40	882.33	1.76	0.08%	0.27
21	2048	optimized	151.922	$9.97\times$	678.74	341.24	1.99	1.08%	4.74
21	4096	naive	6076.821	$1.00\times$	48937.54	13875.13	3.53	0.02%	0.06
21	4096	reorder	3522.121	$1.73\times$	52996.25	8387.40	6.32	0.03%	0.07
21	4096	unroll	1919.387	$3.17\times$	13647.89	4912.54	2.78	0.03%	0.20
21	4096	tile	6087.783	$1.00\times$	49293.82	13960.91	3.53	0.03%	0.09
21	4096	simd	2038.493	$2.98\times$	6201.54	3521.60	1.76	0.08%	0.26
21	4096	optimized	605.269	$10.04\times$	2711.08	1364.08	1.99	1.09%	4.79



(a) Speedup vs. Matrix Size at $K = 3$



(b) Speedup vs. Kernel Size at $N = 2048$

Figure 8: Task 1D: Performance scaling of all optimization stages: (a) Speedup over naive baseline across matrix dimensions ($K = 3$), and (b) Speedup across kernel sizes ($N = 2048$).

7 Github repo link

[here we go](#)